

claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_00bebd3c-82f\agent-a334168666b0ed621.jsonl`
- SHA-256: `fa88d6cb74463ccba049d4135c710e3ec495f3557c8d0efa1311441316063f`
- Source modified: `2026-07-08T05:50:09+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_00bebd3c-82f`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`
```

Transcript

1. user (2026-07-08T05:44:49.638Z)

You are mapping a subsystem of the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (Windows checkout; read-only ? do NOT edit anything).
Context: GitHub issue #524 wants to (a) move validation onto the L4 Cloud Run Job worker via a config-driven phase_placement knob, (b) drop a redundant re-hash in /api/process by trusting the upload's streamed video_id, and (c) design a topology where upload->stitch runs on the L4 with async copy-to-bucket. You are producing the precise code map a designer/implementer needs. Cite exact file paths and line numbers. Read the actual code, not just docs. Follow imports where the flow crosses files.
Subsystem: COMPUTE BACKENDS + WORKER. Map backend/compute/ (base ABC, registry, local + cloud_run implementations) and the worker entrypoint that runs inside the rally-worker Cloud Run Job (find where "[worker] pulled" is logged). Detail: how a dispatch is encoded (job args? env? manifest file in GCS?), how the worker pulls the input, the #513 skip-revalidation flag (how the worker knows CP already validated ? exact flag/field name), what the worker uploads back (done-marker, results manifest), how the CP polls/detects completion, and the #515 crash-before-marker fast-fail. Note what it would take for the worker to RUN validation and report a verdict (rejected video) back to the CP ? does the result manifest have room for a validation verdict/error, and how do worker failures surface to the job today?

2. user (2026-07-08T05:44:58.487Z)

```
backend\compute\base.py
backend\compute\cloud_run.py
backend\compute\__init__.py
backend\compute\__pycache__\base.cpython-313.pyc
backend\compute\__pycache__\__init__.cpython-313.pyc
backend\compute\__pycache__\cloud_run.cpython-313.pyc
```

3. user (2026-07-08T05:44:59.413Z)

```
backend\worker\__main__.py:268:    print(f"[worker] pulled {args.video_uri} -> {local_video}")
```

4. user (2026-07-08T05:45:05.065Z)

```
1      """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5      core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6      (cloud-serving) ? without the core ever branching on the profile (D1).
7
8      The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9      non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
```

```

10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Callable, Optional
18
19
20     class RemoteExecutionFailed(RuntimeError):
21         """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22         completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23         an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24         Succeeded run.
25
26         Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be
27         running): this says the execution is DEFINITELY over and no marker will ever
appear, so the
28         dispatcher must stop bucket-polling immediately instead of burning the full
29         ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30         job + execution name + terminal state + the ``gcloud`` command to read the execution
log, so
31         the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32
33
34     @dataclass(frozen=True)
35     class ComputeJobSpec:
36         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
37         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
38         deterministic regardless of where the job ran."""
39
40         video_uri: str # input (gs://? for cloud; a path/local:// for local)
41         out_uri: str # output ROOT (gs://? bucket for cloud)
42         segmenter: Optional[str] = None
43         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
44         config_overrides: tuple[str, ...] = ()
45         # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving
control-plane) is
46         # the authoritative validation gate: it runs the SAME validators with ITS resolved
config and only
47         # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
decode, and
48         # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49         # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50         # validating exactly as before.
51         skip_validation: bool = False
52
53
54     @dataclass(frozen=True)
55     class ComputeResult:
56         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code

```

```

57         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed.""
60
61         returncode: int
62         outputs_uri: str = ""
63         video_id: str = ""
64         report: dict = field(default_factory=dict)
65         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66         # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67         # executions describe <execution>). Empty for the in-process path / backends without
one.
68         execution: str = ""
69
70
71     class ComputeBackend(ABC):
72         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
73         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74         execution identically."""
75
76         @abstractmethod
77         def run_infer(
78             self,
79             spec: ComputeJobSpec,
80             *,
81             on_wait: "Callable[[float], None] | None" = None,
82         ) -> ComputeResult:
83             """Run ``spec`` to completion and return its ``ComputeResult``.
84
85             ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86             still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
87             long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
88

```

5. user (2026-07-08T05:45:05.542Z)

```

1         """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2         for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4         Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6             --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8             completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9             it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10            cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11            job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12            worker that finished just past the deadline is rescued, not wasted ? then best-

```

```

effort CANCELS
13         the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14         *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15         Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16         worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17         (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18         3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19         ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21         The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22         so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24         The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
25         with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26         owner-verified on the M7 real run.
27         ""
28
29         from __future__ import annotations
30
31         import json
32         import logging
33         import os
34         import tempfile
35         import uuid
36         from abc import ABC, abstractmethod
37         from typing import Any, Callable, List, Optional
38
39         from backend.compute.base import (
40             ComputeBackend,
41             ComputeJobSpec,
42             ComputeResult,
43             RemoteExecutionFailed,
44         )
45         from backend.infrastructure.execution_policy import (
46             DEFAULT_POLICY,
47             ExecutionPolicy,
48             PolicyTimeout,
49             poll_until,
50         )
51         from backend.storage import fetch, get_storage, parse_uri
52
53         LOG = logging.getLogger("compute.cloud_run")
54
55         # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
56         # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
57         _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
58             "deployment.compute_target=local_gpu",
59             "indexing.detector_impl=native",
60         )
61
62
63         class CloudRunJobClient(ABC):
64             """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.

```

```

65
66         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
67         overrides the job's container ``args`` for this execution and starts it."""
68
69         @abstractmethod
70         def run_job(
71             self,
72             job_name: str,
73             argv: List[str],
74             *,
75             region: str,
76             project: Optional[str] = None,
77         ) -> str:
78             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
79             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
80
81         @abstractmethod
82         def cancel_execution(self, execution: str) -> None:
83             """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
84             times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
85             May raise ? the caller treats every failure as best-effort (the original poll
timeout is
86             NEVER masked by a cancel error)."""
87
88         def execution_terminal_state(self, execution: str) -> Optional[str]:
89             """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
90             ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
91             can't be determined).
92
93             ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94             terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95             of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97             NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98             chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99             polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101             return None
102
103
104         class GoogleCloudRunJobClient(CloudRunJobClient):
105             """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106             a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108             def run_job(
109                 self,
110                 job_name: str,
111                 argv: List[str],
112                 *,

```

```

113         region: str,
114         project: Optional[str] = None,
115     ) -> str:
116         # A None/empty project would build a bogus "projects/None/locations/..." path
117         (job_path
118         # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
119         exercisable
120         # in CI (where google-cloud-run is absent) and so mypy narrows project to str
121         for job_path.
122         if not project:
123             raise RuntimeError(
124                 "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
125                 project id "
126                 "to resolve the job path; set it on the control plane (see
127                 get_compute_backend)."
128             )
129         try:
130             from google.cloud import run_v2 # type: ignore[attr-defined]
131             except Exception as exc: # pragma: no cover - real SDK not present in CI
132                 raise RuntimeError(
133                     "google-cloud-run is required for CloudRunCompute's real client; install
134                     it on the "
135                     "control plane (it is intentionally NOT a CI/test dependency)."
136                 ) from exc
137         client = (
138             run_v2.JobsClient()
139         ) # pragma: no cover - exercised only on the real M7 run
140         name = client.job_path(project, region, job_name)
141         overrides = run_v2.RunJobRequest.Overrides(
142             container_overrides=[
143                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
144             ]
145         )
146         op = client.run_job(
147             request=run_v2.RunJobRequest(name=name, overrides=overrides)
148         )
149         return (
150             getattr(op, "metadata", None)
151             and getattr(op.metadata, "name", "")
152             or "execution"
153         )
154
155     def cancel_execution(self, execution: str) -> None:
156         # run_job falls back to the literal string "execution" when the operation
157         metadata carries
158         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
159         import
160         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
161         (where
162         # google-cloud-run is absent).
163         if "/executions/" not in (execution or ""):
164             raise RuntimeError(
165                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
166                 expected "
167                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
168                 resolve "
169                 "one from the operation metadata, so there is nothing to cancel by
170                 name)."
171             )
172         try:
173             from google.cloud import run_v2 # type: ignore[attr-defined]
174             except Exception as exc: # pragma: no cover - real SDK not present in CI
175                 raise RuntimeError(
176                     "google-cloud-run is required to cancel a Cloud Run execution; install
177                     it on the "

```

```

165         "control plane (it is intentionally NOT a CI/test dependency)."
166     ) from exc
167     client = (
168         run_v2.ExecutionsClient()
169     ) # pragma: no cover - exercised only on a real run
170     client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
171
172     def execution_terminal_state(self, execution: str) -> Optional[str]:
173         # Advisory status probe (see the ABC docstring): fetch the execution and
174         # pure, unit-tested ``_classify_execution_state``. A non-resolvable name /
175         # absent SDK returns
176         # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
177         # raises on bad
178         # input, because the probe is best-effort and must not fail a healthy run.
179         if "/executions/" not in (execution or ""):
180             return None
181         try:
182             from google.cloud import run_v2 # type: ignore[attr-defined]
183         except Exception: # pragma: no cover - real SDK not present in CI
184             return None
185         client = (
186             run_v2.ExecutionsClient()
187         ) # pragma: no cover - exercised only on a real run
188         exe = client.get_execution( # pragma: no cover - real run only
189             request=run_v2.GetExecutionRequest(name=execution)
190         )
191         return _classify_execution_state(exe) # pragma: no cover - real run only
192
193     def _classify_execution_state(exe: Any) -> Optional[str]:
194         """Map a Cloud Run ``run_v2.Execution`` to a terminal state label (``"Failed"`` /
195         ``"Cancelled"``
196         / ``"Succeeded"``), or ``None`` if it is still pending/running. PURE (no
197         SDK/network) so it is
198         unit-tested against both real proto-plus ``Execution`` objects and lightweight fakes
199         ? the actual
200         ``get_execution`` fetch is the only part that can't run in CI.
201         ``google-cloud-run`` is a **proto-plus** library, so a ``google.protobuf.Timestamp``
202         field is
203         marshalled to a native Python ``datetime`` when set and to ``None`` when unset ? NOT
204         to a proto
205         ``Timestamp`` with a ``.seconds`` field (a ``datetime`` has ``.second``, singular).
206         Terminality is
207         therefore ``completion_time`` is not None; the per-task counts (plain ints) then
208         classify it,
209         with a cancel taking precedence (a cancelled execution reports ``cancelled_count >
210         0``)."""
211         if getattr(exe, "completion_time", None) is None:
212             return None # no terminal completion timestamp yet ? still pending/running
213         if getattr(exe, "cancelled_count", 0):
214             return "Cancelled"
215         if getattr(exe, "failed_count", 0):
216             return "Failed"
217         if getattr(exe, "succeeded_count", 0):
218             return "Succeeded"
219         return "Failed" # terminal but no positive success ? treat as failed (conservative)
220
221     class CloudRunCompute(ComputeBackend):
222         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
223
224         def __init__(
225             self,

```

```

219         *,
220         run_client: CloudRunJobClient,
221         job_name: str,
222         region: str,
223         project: Optional[str] = None,
224         storage_config: Optional[dict] = None,
225         policy: ExecutionPolicy = DEFAULT_POLICY,
226         token: Optional[str] = None,
227         container_overrides: Optional[tuple[str, ...]] = None,
228     ) -> None:
229         self._client = run_client
230         self._job_name = job_name
231         self._region = region
232         self._project = project
233         self._storage_config = storage_config or {}
234         self._policy = policy
235         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
236         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
237         self._token = token
238         self._container_overrides = (
239             _DEFAULT_CONTAINER_OVERRIDES
240             if container_overrides is None
241             else tuple(container_overrides)
242         )
243
244     def run_infer(
245         self,
246         spec: ComputeJobSpec,
247         *,
248         on_wait: "Callable[[float], None] | None" = None,
249     ) -> ComputeResult:
250         out_root = spec.out_uri.rstrip("/")
251         token = self._token or uuid.uuid4().hex
252         done_uri = f"{out_root}/_dispatch/{token}.done"
253         argv: List[str] = [
254             "infer",
255             "--video-uri",
256             spec.video_uri,
257             "--out-uri",
258             spec.out_uri,
259             "--done-uri",
260             done_uri,
261         ]
262         if spec.segmenter:
263             argv += ["--segmenter", spec.segmenter]
264         if spec.skip_validation:
265             # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
266             # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
267             argv.append("--skip-validation")
268         for kv in (*spec.config_overrides, *self._container_overrides):
269             argv += ["--set", kv]
270
271         execution = self._client.run_job(
272             self._job_name, argv, region=self._region, project=self._project
273         )
274         LOG.info(
275             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
276             self._job_name,
277             execution,
278             done_uri,
279         )

```



```

280
281         try:
282             marker = poll_until(
283                 lambda: self._poll_check(done_uri, str(execution or "")),
284                 self._policy,
285                 label="cloud-run-infer",
286                 logger=LOG,
287                 # Live heartbeat (#482): each still-waiting tick reaches the caller so
288                 # job.progress moves during the whole GPU run instead of freezing.
289                 on_tick=on_wait,
290             )
291         except PolicyTimeout as timeout_exc:
292             # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
293             # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
294             # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
295             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
296             video_id = str(marker.get("video_id", ""))
297             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
298             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
299             rc = int(marker.get("returncode", 1))
300             if not video_id:
301                 LOG.warning(
302                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
303                     "video_id) ? treating as failure",
304                     self._job_name,
305                 )
306             rc = rc or 1
307             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
308             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
309             LOG.info(
310                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
311             )
312             return ComputeResult(
313                 returncode=rc,
314                 outputs_uri=outputs_uri,
315                 video_id=video_id,
316                 report=report,
317                 execution=str(execution or ""),
318             )
319
320     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
321         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
322
323         The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
324         wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
325         FAILURE* marker resolves through the normal path with its real returncode. Only
when the
326         marker is absent do we consult the execution status: a worker that TERMINATED
without ever
327         writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
328         otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
329         opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
330         failure.
331

```

```

332         Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
333         ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
334         will ever appear)."""
335         marker = self._read_marker(done_uri)
336         if marker is not None:
337             return marker # success signal ? authoritative even if status later flips
338         state = self._terminal_execution_state(execution)
339         if state is None:
340             return None # still pending/running, or status unknown ? keep bucket-
polling
341         # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
342         # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
343         # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
344         # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
345         marker = self._read_marker(done_uri)
346         if marker is not None:
347             return marker
348         raise RemoteExecutionFailed(
349             f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
350             f"terminal state {state!r} with NO completion marker ? the worker exited
before "
351             "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
352             "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
353             "inspect the Cloud Run Job execution log to diagnose: "
354             f"`gcloud run jobs executions describe {execution or '<execution>'} "
355             f"--region {self._region}`."
356         )
357
358     def _terminal_execution_state(self, execution: str) -> Optional[str]:
359         """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
360         flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
361         so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
362         ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
363         if not execution:
364             return None
365         try:
366             return self._client.execution_terminal_state(execution)
367         except Exception: # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
368             LOG.debug(
369                 "cloud-run: execution status probe failed for %s ? treating as unknown",
370                 execution,
371                 exc_info=True,
372             )
373             return None
374
375     def _handle_poll_timeout(
376         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
377     ) -> dict:
378         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
379
380         The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
381         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past

```

```

the deadline:
382         ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
383         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
384         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
385         execution name so the operator can verify the state in the GCP console. A cancel
failure
386         NEVER masks the original timeout."""
387         try:
388             late = self._read_marker(done_uri)
389         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
390             LOG.warning(
391                 "cloud-run: final post-timeout marker check failed ? treating as
absent",
392                 exc_info=True,
393             )
394             late = None
395         if late is not None:
396             LOG.warning(
397                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
398                 "? rescuing the completed result",
399                 self._job_name,
400                 execution,
401             )
402             return late
403         try:
404             self._client.cancel_execution(execution)
405         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
406             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
407             LOG.warning(
408                 "cloud-run: best-effort cancel of execution=%s failed: %s",
409                 execution,
410                 cancel_exc,
411                 exc_info=True,
412             )
413         else:
414             cancel_note = "cancel requested"
415             LOG.info("cloud-run: requested cancel of execution=%s", execution)
416             raise PolicyTimeout(
417                 f"{timeout_exc} [cloud-run job={self._job_name} "
418                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
419                 "state in the GCP console]"
420             ) from timeout_exc
421
422         # --- storage-backed completion signal (bucket-poll) ---
423         def _read_marker(self, done_uri: str) -> Optional[dict]:
424             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
425             ref = parse_uri(done_uri)
426             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
427                 return None
428             return (
429                 self._read_json(done_uri) or {}
430             ) # empty dict still terminates the poll (present == done)
431
432         def _read_json(self, uri: str) -> dict:
433             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
434             # raises ValueError ? unlike local's identity passthrough. Without an explicit

```

```

dst this read
435         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
436         try:
437             ref = parse_uri(uri)
438             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
439                 dst = os.path.join(td, ref.name or "obj.json")
440                 local = fetch(uri, dst, config=self._storage_config)
441                 with open(local, "r", encoding="utf-8") as f:
442                     data = json.load(f)
443                 return data if isinstance(data, dict) else {}
444         except Exception:
445             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
446         return {}
447

```

6. user (2026-07-08T05:45:07.295Z)

```

1         """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3         python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4             [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5         python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7         The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8         is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9         are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10        ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11        the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13        This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14        (validator registry ? segmenter ? report), never a forked pipeline.
15        """
16
17        from __future__ import annotations
18
19        import argparse
20        import csv
21        import json
22        import logging
23        import os
24        import re
25        import sys
26        import tempfile
27        from typing import Any, List
28
29        from backend import storage
30        from backend.compute import ComputeJobSpec, get_compute_backend
31        from backend.config import (
32            StartupCheckError,
33            enforce_startup_checks,
34            load_config,
35            probe_cuda_available,
36        )
37        from backend.infrastructure.database import Database
38        from backend.pipeline.segmenters.base import segmenter_registry
39        from backend.pipeline.validators.base import registry as validator_registry
40        from backend.reporting import emit_indexer_report
41        from backend.results import Failure
42        from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43        from backend.utils.hashing import compute_video_id

```

```

44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50         str."""
51         low = v.lower()
52         if low in ("true", "false"):
53             return low == "true"
54         for cast in (int, float):
55             try:
56                 return cast(v)
57             except ValueError:
58                 pass
59         return v
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
63         indexing.skip_ai_handoff=true)."""
64         for kv in sets or []:
65             key, sep, val = kv.partition("=")
66             if not sep:
67                 raise ValueError(f"--set expects k=v, got {kv!r}")
68             parts = key.split(".")
69             obj = config
70             for p in parts[:-1]:
71                 obj = getattr(obj, p)
72                 setattr(obj, parts[-1], _coerce(val))
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
76         friendly
77         artifact (same schema as the rally-annotator labels)."""
78         with open(path, "w", newline="") as f:
79             w = csv.writer(f)
80             w.writerow(["start", "end", "shot_count", "ending_reason"])
81             for s in segments:
82                 w.writerow(
83                     [
84                         f"{float(s.get('start_time', 0.0)):.3f}",
85                         f"{float(s.get('end_time', 0.0)):.3f}",
86                         s.get("shot_count", ""),
87                         s.get("ending_reason", s.get("shot_count_confidence", "")),
88                     ]
89                 )
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {

```

```

106         "start": float(s.get("start_time", 0.0)),
107         "end": float(s.get("end_time", 0.0)),
108     }
109     for s in segments
110 ],
111 },
112 f,
113     indent=2,
114 )
115
116
117 def _serialize_and_push_outputs(
118     scratch: str,
119     out_uri: str,
120     video_id: str,
121     segments: List[dict],
122     status: str,
123     storage_cfg: dict,
124     video_uri: str = "",
125 ) -> List[str]:
126     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
127     push both to
128     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
129
130     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
131     timeout/abort leaves a
132     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
133     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
134     non-``success``
135     report as "no usable result" ? retry, and the restart-recovery poll waits for
136     report.json to
137     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
138     """
139     out_local = os.path.join(scratch, "outputs", video_id)
140     os.makedirs(out_local, exist_ok=True)
141     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
142     _write_report_json(
143         video_id,
144         segments,
145         status,
146         os.path.join(out_local, "report.json"),
147         video_uri=video_uri,
148     )
149     out_prefix = out_uri.rstrip("/")
150     pushed: List[str] = []
151     for fn in sorted(os.listdir(out_local)):
152         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154         pushed.append(uri)
155     return pushed
156
157
158 def _run_startup_checks(config: Any) -> bool:
159     """M5 per-profile startup checks for the worker (the compute process, so it probes
160     CUDA
161     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
162     aborts
163     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
164     no-op
165     (returns True, ZERO work) when no deployment.profile is selected.
166
167     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
168     imports
169     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
170     delegated to a
171     WSL/native subprocess). Probing only when a profile is set keeps the default-OFF

```

```

path a true
162     no-op of work, not just of output."""
163     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164     cuda = (
165         probe_cuda_available() if profile else None
166     ) # torch-free on the default-OFF path
167     try:
168         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169         return True
170     except StartupCheckError:
171         return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181         * rc 4 ? the bucket poll exhausted its budget (`PolicyTimeout`): the execution
may still be
182         running (the shim best-effort cancels it).
183         * rc 5 ? the execution has TERMINATED without a marker
(`RemoteExecutionFailed`): a worker
184         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185         ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
214         return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,

```

```

219     video_id: str,
220     returncode: int,
221     segment_count: int,
222     status: str,
223     storage_cfg: dict,
224     scratch: str,
225 ) -> None:
226     """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227     dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228     try:
229         marker = {
230             "video_id": video_id,
231             "returncode": returncode,
232             "segment_count": segment_count,
233             "status": status,
234         }
235         local = os.path.join(scratch, "_done.json")
236         with open(local, "w", encoding="utf-8") as f:
237             json.dump(marker, f)
238         storage.put(local, done_uri, config=storage_cfg)
239         logger.info("[worker] wrote completion marker -> %s", done_uri)
240     except Exception as e: # never fail a good run because the marker push hiccuped
241         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
242
243
244 def cmd_infer(args: argparse.Namespace) -> int:
245     config = load_config(args.config) if args.config else load_config()
246     _apply_overrides(config, args.set)
247     if not _run_startup_checks(config):
248         return 2
249
250     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253     # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254     compute = get_compute_backend(config)
255     if compute is not None:
256         return _dispatch_remote(compute, args)
257
258     storage_cfg = config.storage.model_dump()
259
260     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261     os.makedirs(scratch, exist_ok=True)
262     config.output.output_dir = scratch
263
264     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265     local_video = storage.fetch(
266         args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267     )
268     print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271     video_id = compute_video_id(local_video)
272
273     # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274     # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the

```



```

275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )
287             val_results, val_context = [], {}
288         else:
289             val_results, val_context = validator_registry.run_all_with_context(
290                 local_video, config
291             )
292         failures = [r for r in val_results if not r.passed]
293         db = Database(os.path.join(scratch, config.output.db_name))
294         db.add_video(
295             video_id,
296             local_video,
297             "failed" if failures else "processed",
298             [r.model_dump() for r in val_results],
299         )
300
301         def _fail(rc: int) -> int:
302             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309             try:
310                 _serialize_and_push_outputs(
311                     scratch,
312                     args.out_uri,
313                     video_id,
314                     [],
315                     "failed",
316                     storage_cfg,
317                     str(getattr(args, "video_uri", "")) or "",
318                 )
319             except Exception as e: # never mask the real failure on an artifact-push hiccup
320                 logger.warning("[worker] could not push failure artifacts: %s", e)
321             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322             if getattr(args, "done_uri", None):
323                 _write_done_marker(
324                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch

```

```

325         )
326         return rc
327
328     segments: List[dict] = []
329     segmenter_actual = None
330     segmentation_ran = False
331     status = "failed"
332     try:
333         if failures:
334             print(
335                 "[worker] validation FAILED: "
336                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337             )
338             return _fail(2)
339         seg_name = args.segmenter or config.indexing.default_segmenter
340         segmenter_cls = segmenter_registry.get(seg_name)
341         if not segmenter_cls:
342             print(
343                 f"[worker] unknown segmenter: {seg_name!r} "
344                 f"(have {list(segmenter_registry.list_available())})"
345             )
346             return _fail(2)
347         segmenter_actual = seg_name
348         result = segmenter_cls(db, config).process_video(
349             video_id, local_video, max_frames=args.max_frames
350         )
351         segmentation_ran = True
352         if isinstance(result, Failure):
353             print(f"[worker] segmenter FAILED: {result.message}")
354             return _fail(3)
355         segments = result.value if hasattr(result, "value") else result
356         status = "success"
357         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360         if not segments:
361             detector_impl = (
362                 os.environ.get("RALLY_DETECTOR_IMPL")
363                 or getattr(config.indexing, "detector_impl", None)
364                 or "auto"
365             )
366             logger.warning(
367                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370                 "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371                 "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372                 "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373                 "detections, or the input resolution differs from the tuned proxy
resolution. "
374                 "Re-run with -v for the native command + frame counts.",
375                 video_id,
376                 segmenter_actual,
377                 detector_impl,
378             )
379         finally:
380             # Best-effort per-video observability report (never raises) ? parity with the

```

```

main CLI.
381         emit_indexer_report(
382             db=db,
383             video_id=video_id,
384             video_path=local_video,
385             config=config,
386             val_results=val_results,
387             val_context=val_context,
388             segmenter_requested=args.segmenter,
389             segmenter_actual=segmenter_actual,
390             was_reingest=False,
391             segmentation_ran=segmentation_ran,
392         )
393
394     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395     pushed = _serialize_and_push_outputs(
396         scratch,
397         args.out_uri,
398         video_id,
399         segments,
400         status,
401         storage_cfg,
402         str(getattr(args, "video_uri", "") or ""),
403     )
404
405     print(
406         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
407     )
408     for u in pushed:
409         print("    ", u)
410
411     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
412     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
413     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
414     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
415     if getattr(args, "done_uri", None):
416         _write_done_marker(
417             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418         )
419     return 0
420
421
422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
424     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
427     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
428     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
430
431
432     def _label_clip_name(fname: str) -> str:
433         """Derive the clip name (the `videos/<name>.<ext>` join key) from a label
filename.
434

```

```

435 Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
436 canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
437
438 * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
439 * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
440 * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
441 * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
442
443 Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
444 video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
445 """
446 name = fname.replace(".rallies.csv", "").replace(".csv", "")
447 return _LABEL_DATE_PREFIX.sub("", name)
448
449
450 def _probe_video_fps_width(path: str):
451     """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
452
453     The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
454     fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
455     cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
456     inference report (it flags the fallback) but corrupting for REAL training data. So
here we
457     probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
458     rather than guesses.
459     """
460     import json
461     import subprocess
462
463     try:
464         out = subprocess.check_output(
465             [
466                 ffprobe_bin(),
467                 "-v",
468                 "error",
469                 "-select_streams",
470                 "v:0",
471                 "-show_entries",
472                 "stream=r_frame_rate,width",
473                 "-of",
474                 "json",
475                 path,
476             ],
477             stderr=subprocess.DEVNULL,
478         ).decode()
479         st = (json.loads(out).get("streams") or [{}])[0]
480         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
481         fps = float(num) / float(den or "1")
482         width = float(st.get("width") or 0)
483         if fps > 0 and width > 0:
484             return fps, width
485     except (
486         subprocess.SubprocessError,
487         ValueError,

```

```

488         KeyError,
489         OSError,
490         json.JSONDecodeError,
491     ):
492         pass
493     return None
494
495
496 def _resolve_train_detector(args: argparse.Namespace, config: Any):
497     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
498     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
499     stub=CPU mock). Returns the runner, or prints an error + returns None.
500
501     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
502     the
503     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
504     has
505     no shuttle detector and is rejected.
506     """
507     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
508
509     seg_cls = segmenter_registry.get(args.segmenter)
510     if not seg_cls:
511         print(
512             f"[worker] unknown segmenter: {args.segmenter!r} "
513             f"(have {list(segmenter_registry.list_available())})"
514         )
515         return None
516     seg = seg_cls(None, config)
517     if not isinstance(seg, TrajectoryHybridSegmenter):
518         print(
519             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
520             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
521         )
522         return None
523     try:
524         runner, _ = seg._resolve_runner(config.indexing)
525     except NotImplementedError as e:
526         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
527         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
528         traceback.
529         print(f"[worker] detector not available: {e}")
530         return None
531     ok, msg = runner.healthcheck()
532     if not ok:
533         print(f"[worker] detector environment not ready: {msg}")
534         return None
535     print(f"[worker] detector ready ({args.segmenter}): {msg}")
536     return runner
537
538
539 def cmd_train(args: argparse.Namespace) -> int:
540     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
541     to training.gen0.harness (backend must NOT import training) -> push the artifact
542     bundle.
543
544     Two trajectory sources (the train pipeline + harness are identical either way):
545     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
546     synthetic
547         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
548     CI.
549     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
550     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).

```

This

```
545         is the M3.5 "first real training" path; only the trajectory source flips.
546         """
547         import subprocess
548
549         config = load_config(args.config) if args.config else load_config()
550         _apply_overrides(config, args.set)
551         if not _run_startup_checks(config):
552             return 2
553         storage_cfg = config.storage.model_dump()
554
555         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
556         labels_dir = os.path.join(scratch, "labels")
557         traj_dir = os.path.join(scratch, "trajectories")
558         videos_dir = os.path.join(scratch, "videos")
559         os.makedirs(labels_dir, exist_ok=True)
560         os.makedirs(traj_dir, exist_ok=True)
561
562         corpus = args.corpus_uri.rstrip("/")
563         label_uris = sorted(
564             u
565             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
566             if u.lower().endswith(".csv")
567         )
568         if len(label_uris) < 2:
569             print(
570                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
571                 f"under {corpus}/labels/"
572             )
573             return 2
574
575         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
576         runner = None
577         video_by_stem: dict = {}
578         if args.trajectory_source == "detector":
579             os.makedirs(videos_dir, exist_ok=True)
580             runner = _resolve_train_detector(args, config)
581             if runner is None:
582                 return 2
583             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
584                 vname = storage.parse_uri(vuri).name
585                 if not vname.lower().endswith(_VIDEO_EXTS):
586                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
587                 video_by_stem[os.path.splitext(vname)[0]] = vuri
588
589         print(f"[worker] trajectory source: {args.trajectory_source}")
590         specs: List[dict] = []
591         for uri in label_uris:
592             fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
593             name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
594             local_label = storage.fetch(
595                 uri, os.path.join(labels_dir, fname), config=storage_cfg
596             )
597             if args.trajectory_source == "detector":
598                 vuri = video_by_stem.get(name)
599                 if not vuri:
600                     print(
601                         f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
602                     )
603                     continue
604                 local_video = storage.fetch(
605                     vuri,
```

```

606         os.path.join(videos_dir, storage.parse_uri(vuri).name),
607         config=storage_cfg,
608     )
609     video_id = compute_video_id(local_video)
610     # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
611     # wrong fps silently mis-aligns every label, so skip the video if ffmpeg
can't read it.
612     meta = _probe_video_fps_width(local_video)
613     if meta is None:
614         print(
615             f"[worker] could not probe fps/width for {name!r} (ffmpeg) ?
skipping (won't guess)."
616         )
617         continue
618     fps, frame_width = meta
619     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
620     if not traj:
621         print(
622             f"[worker] detector produced no trajectory for {name!r} ? skipping."
623         )
624         continue
625     specs.append(
626         {
627             "name": name,
628             "trajectory": traj,
629             "labels": local_label,
630             "fps": fps,
631             "frame_width": frame_width,
632         }
633     )
634     else:
635         from backend.pipeline.detectors.stub_runner import (
636             synth_trajectory_from_labels,
637         )
638
639         traj = os.path.join(traj_dir, f"{name}_traj.csv")
640         synth_trajectory_from_labels(
641             local_label, traj, fps=args.fps, frame_width=args.frame_width
642         )
643         specs.append(
644             {
645                 "name": name,
646                 "trajectory": traj,
647                 "labels": local_label,
648                 "fps": args.fps,
649                 "frame_width": args.frame_width,
650             }
651         )
652
653     if len(specs) < 2:
654         print(
655             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
656         )
657         return 2
658     manifest_path = os.path.join(scratch, "manifest.json")
659     with open(manifest_path, "w", encoding="utf-8") as f:
660         json.dump(specs, f, indent=2)
661     src_label = (
662         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
663     )
664     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
665
666     out_dir = os.path.join(scratch, "artifacts")

```

```

667     cmd = [
668         sys.executable,
669         "-m",
670         "training.gen0.harness",
671         "--manifest",
672         manifest_path,
673         "--out",
674         out_dir,
675         "--version",
676         args.version,
677     ]
678     if args.floor:
679         floor_local = (
680             storage.fetch(
681                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
682             )
683             if "://" in args.floor
684             else args.floor
685         )
686         cmd += ["--floor", floor_local]
687     print("[worker] training:", " ".join(cmd))
688     rc = subprocess.run(cmd).returncode
689     if rc != 0:
690         print(f"[worker] gen0 harness failed (rc={rc})")
691         return rc
692
693     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
694     bundle = os.path.join(out_dir, args.version)
695     out_prefix = args.out_uri.rstrip("/")
696     pushed = []
697     for fn in sorted(os.listdir(bundle)):
698         uri = f"{out_prefix}/weights/{args.version}/{fn}"
699         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
700         pushed.append(uri)
701     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
702     for u in pushed:
703         print("    ", u)
704     return 0
705
706
707 def build_parser() -> argparse.ArgumentParser:
708     p = argparse.ArgumentParser(
709         prog="python -m backend.worker",
710         description="Storage-aware worker: pull ? run pipeline ? push.",
711     )
712     p.add_argument(
713         "-v",
714         "--verbose",
715         action="store_true",
716         help="DEBUG logging (the native detector command, frame counts, per-stage
717         detail)",
718     )
719     sub = p.add_subparsers(dest="cmd", required=True)
720
721     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
722     pi.add_argument(
723         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
724     )
725     pi.add_argument(
726         "--out-uri",
727         required=True,
728         help="output prefix (outputs/<video_id>/? is appended)",
729     )
730     pi.add_argument(
731         "--segmenter", default=None, help="default: indexing.default_segmenter"

```



```

731     )
732     pi.add_argument(
733         "--config", default=None, help="config.json path (default: ./config.json)"
734     )
735     pi.add_argument(
736         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
737     )
738     pi.add_argument("--max-frames", type=int, default=None)
739     pi.add_argument(
740         "--done-uri",
741         default=None,
742         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
743         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
744         "Default-OFF: omit it for the normal local run.",
745     )
746     pi.add_argument(
747         "--set",
748         action="append",
749         default=[],
750         metavar="k=v",
751         help="config override, e.g. --set indexing.skip_ai_handoff=true",
752     )
753     pi.add_argument(
754         "--skip-validation",
755         action="store_true",
756         help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
757         "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
758         "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
759         "`worker infer` CLI still validates.",
760     )
761     pi.set_defaults(func=cmd_infer)
762
763     pt = sub.add_parser(
764         "train",
765         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
766     )
767     pt.add_argument(
768         "--corpus-uri",
769         required=True,
770         help="prefix containing labels/ (+ videos/ for "
771         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
772     )
773     pt.add_argument(
774         "--out-uri",
775         required=True,
776         help="output prefix (weights/<version>/ is appended)",
777     )
778     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
779     pt.add_argument(
780         "--trajectory-source",
781         choices=["synth", "detector"],
782         default="synth",
783         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
784         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
785         "(needs videos/ in the corpus; honours detector_impl).",
786     )
787     pt.add_argument(
788         "--segmenter",
789         default="wasb_hybrid",
790         help="trajectory hybrid whose detector is used for --trajectory-source
detector",
791     )

```

```

792     pt.add_argument(
793         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
794     )
795     pt.add_argument("--config", default=None)
796     pt.add_argument("--scratch", default=None)
797     pt.add_argument(
798         "--fps",
799         type=float,
800         default=30.0,
801         help="synth trajectory fps (detector probes the video)",
802     )
803     pt.add_argument(
804         "--frame-width",
805         type=float,
806         default=1280.0,
807         help="synth trajectory frame width (detector probes the video)",
808     )
809     pt.add_argument("--set", action="append", default=[], metavar="k=v")
810     pt.set_defaults(func=cmd_train)
811     return p
812
813
814     def _setup_logging(verbose: bool) -> None:
815         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
816         diagnostics are
817         VISIBLE ? the worker previously ran with no logging config, so every
818         segmenter/detector log
819         (detector choice, 'Running native WASB inference', availability failures) was
820         silently dropped.
821         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
822         (the CLI
823         entry), not at import, so importing the module never reconfigures a caller's
824         logging."""
825         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
826         library
827         # configured logging at import), which would leave the level at WARNING and re-
828         suppress the
829         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
830         verbosity.
831         logging.basicConfig(
832             level=logging.DEBUG if verbose else logging.INFO,
833             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
834             datefmt="%H:%M:%S",
835             force=True,
836         )
837
838     def main(argv=None) -> int:
839         args = build_parser().parse_args(argv)
840         _setup_logging(getattr(args, "verbose", False))
841         return args.func(args)
842
843     if __name__ == "__main__":
844         sys.exit(main())
845

```

7. user (2026-07-08T05:45:07.381Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5     ``None``
6     otherwise ? ``None`` means "run the worker's existing in-process path" (today's

```

```

behaviour, byte
6     identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
  ``cpu_mock`` nothing
7     changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
  needed).
8     ""
9
10    from __future__ import annotations
11
12    import dataclasses
13    import os
14    from typing import Any, Optional
15
16    from backend.compute.base import (
17        ComputeBackend,
18        ComputeJobSpec,
19        ComputeResult,
20        RemoteExecutionFailed,
21    )
22    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
23
24    __all__ = [
25        "ComputeBackend",
26        "ComputeJobSpec",
27        "ComputeResult",
28        "RemoteExecutionFailed",
29        "get_compute_backend",
30    ]
31
32
33    def _storage_dict(config: Any) -> dict:
34        storage = getattr(config, "storage", None)
35        if storage is None:
36            return {}
37        if hasattr(storage, "model_dump"):
38            return storage.model_dump() # type: ignore[no-any-return]
39        return dict(storage) if isinstance(storage, dict) else {}
40
41
42    def get_compute_backend(
43        config: Any,
44        *,
45        run_client: Optional[Any] = None,
46        policy: ExecutionPolicy = DEFAULT_POLICY,
47        token: Optional[str] = None,
48        target_override: Optional[str] = None,
49    ) -> Optional[ComputeBackend]:
50        """Return the compute backend for ``config.deployment.compute_target``, or ``None``
  to run
51        in-process (the default-OFF path).
52
53        Only ``compute_target == "cloud_run"`` returns a non-None backend (a
  ``CloudRunCompute``). Its
54        Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
  ``CLOUD_RUN_REGION``
55        / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
  injectable (tests
56        pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
  verified on M7).
57
58        ``target_override`` lets a single request force a target (the SPA compute-picker
  maps a
59        per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
  the
60        server's configured default. ``None`` falls back to

```

```

``config.deployment.compute_target``.
61     """
62     target = (
63         target_override
64         if target_override is not None
65         else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
66     )
67     if target != "cloud_run":
68         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
69
70     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
71     # selected.
72     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
73
74     # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
75     # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
76     # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the
77     # old
78     # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
79     # an
80     # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
81     if policy is DEFAULT_POLICY:
82         max_wait = getattr(
83             getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
84         )
85         if max_wait:
86             policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
87
88     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
89     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
90     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
91     client = run_client if run_client is not None else GoogleCloudRunJobClient()
92     return CloudRunCompute(
93         run_client=client,
94         job_name=job_name,
95         region=region,
96         project=project,
97         storage_config=_storage_dict(config),
98         policy=policy,
99         token=token,
100     )

```

8. assistant (2026-07-08T05:45:18.098Z)

Now let me find the control-plane dispatch side (`orchestration.\_maybe\_dispatch\_remote`), the deployment config model, and the execution policy.

9. user (2026-07-08T05:45:19.705Z)

```

backend\api\job_worker.py:137:         # v8 compute-picker: the worker runs _execute_processing ?
backend\api\job_worker.py:137:         _maybe_dispatch_remote, which
backend\api\job_worker.py:364:         # Same normalization the execution path applies
backend\api\job_worker.py:364:         (_maybe_dispatch_remote lowercases
backend\compute\base.py:51:         skip_validation: bool = False
backend\compute\cloud_run.py:122:         "to resolve the job path; set it on the control
backend\compute\cloud_run.py:122:         plane (see get_compute_backend)."
backend\compute\cloud_run.py:264:         if spec.skip_validation:
backend\compute\__init__.py:3: COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is
backend\compute\__init__.py:3: the seam: it returns a
backend\compute\__init__.py:29:         "get_compute_backend",
backend\compute\__init__.py:42: def get_compute_backend(
backend\main.py:241:         _maybe_dispatch_remote,
backend\main.py:268:         # default (deployment.compute_target). Only honoured for these two

```

```

values; see _maybe_dispatch_remote.
backend\worker\_main_.py:30:from backend.compute import ComputeJobSpec, get_compute_backend
backend\worker\_main_.py:194:         skip_validation=bool(getattr(args, "skip_validation",
False)),
backend\worker\_main_.py:251:     # DEFAULT-OFF: get_compute_backend returns None for
"""/local_gpu/cpu_mock ? fall through to the
backend\worker\_main_.py:254:     compute = get_compute_backend(config)
backend\worker\_main_.py:280:     # --skip-validation (set by the dispatcher in
orchestration._maybe_dispatch_remote) skips it; the
backend\worker\_main_.py:282:     if getattr(args, "skip_validation", False):
backend\orchestration.py:4:Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` /
`_ingest_remote_segments`),
backend\orchestration.py:161:# guardrail in _maybe_dispatch_remote fails such a request FAST
instead of the cryptic deep
backend\orchestration.py:423:def _maybe_dispatch_remote(
backend\orchestration.py:441:    (**DEFAULT-OFF**: `_get_compute_backend` returns `None` for
every non-`cloud_run` target, so
backend\orchestration.py:443:        from backend.compute import ComputeJobSpec, get_compute_backend
backend\orchestration.py:506:        compute = get_compute_backend(cfg,
target_override="cloud_run")
backend\orchestration.py:508:        compute = get_compute_backend(
backend\orchestration.py:631:        skip_validation=True,
backend\orchestration.py:777:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran
(validation/config fail).
backend\orchestration.py:906:        remote = _maybe_dispatch_remote(

```

10. user (2026-07-08T05:45:20.354Z)

```

backend\config\loader.py:200:         ct = merged["deployment"].get("compute_target")
backend\config\loader.py:203:         # An active profile with no explicit compute_target
(e.g. config.json blanked it
backend\config\loader.py:205:         # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend\config\loader.py:206:         merged["deployment"]["compute_target"] = canonical
backend\config\loader.py:209:         "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend\config\loader.py:216:         "[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend\config\loader.py:218:         merged["deployment"].get("compute_target"),
backend\config\loader.py:254:# local storage and the compute_target, so this stays the handful
of knobs a fresh, non-technical
backend\config\presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend\config\presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend\config\presets.py:39:# that EXIST today ? M1 is pure/additive: the new
`deployment.{profile,compute_target}` plus
backend\config\presets.py:45:     "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend\config\presets.py:52:     "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend\config\presets.py:75:     "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},
backend\config\models.py:471:     compute_target: Literal["", "local_gpu", "cloud_run",
"cpu_mock"] = ""
backend\config\models.py:473:     # bucket-poll CONCURRENTLY when compute_target=="cloud_run".
Each is an I/O-bound Cloud Run Job
backend\config\models.py:483:     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)

```

11. user (2026-07-08T05:45:25.856Z)

```

380         if not segments:
381             return None
382         logger.info(
383             "[API] re-hydrated %d segments for video_id=%s from %s (submit-time

```

```

cache hit)",
384         n,
385         video_id,
386         outputs_uri,
387     )
388     return _cached_process_payload(video_id, [], segments, requested_compute)
389 except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
390     logger.warning(
391         "[API] submit-time cache check failed for %s ? dispatching normally",
392         video_id,
393         exc_info=True,
394     )
395     return None
396
397
398 def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
399     """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
400     try:
401         return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
402     except Exception: # noqa: BLE001 ? recovery nicety, never a failure
403         return ""
404
405
406 _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
407
408
409 def read_path_rehydrate(db, cfg, video_id: str) -> bool:
410     """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
411     nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
so
412     ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
413     has the video's rows now (re-query). Non-md5 ids return False without touching the
414     bucket; every failure degrades to False (the read just reports what the DB has)."""
415     vid = (video_id or "").strip().lower()
416     if not _MD5_HEX_RE.fullmatch(vid):
417         return False
418     if not probe_process_cache(db, cfg, vid):
419         return False
420     return cached_process_result(db, cfg, vid, "") is not None
421
422
423 def _maybe_dispatch_remote(
424     cfg,
425     db,
426     req: "ProcessRequest",
427     video_id: str,
428     seg_name: str,
429     val_results,
430     play_wm: int,
431     cand_wm: int,
432     notify,
433 ) -> Optional[Tuple[Dict[str, Any], bool]]:
434     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
435     == "cloud_run"`, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
436     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
437
438     Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
439     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
440     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-

```

```

process segmenter
441      (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
442      the in-process path stays byte-identical)."""
443      from backend.compute import ComputeJobSpec, get_compute_backend
444
445      def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
446          # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
447          # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
448          db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
449          # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
450          # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
451          # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
452          # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
453          return (
454              {
455                  "video_id": video_id,
456                  "status": "failed",
457                  "message": msg,
458                  "results": [r.model_dump() for r in val_results],
459                  "play_segments": [],
460                  "compute": {
461                      "path": "cloud_run" if ran else "not_run",
462                      "requested": req.compute,
463                      "target": "cloud_run",
464                      "dispatched": ran,
465                  },
466              },
467              ran,
468          )
469
470          # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471          # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472          # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473          choice = (req.compute or "").strip().lower()
474          if choice and choice not in ("local", "cloud"):
475              logger.warning(
476                  "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477                  req.compute,
478              )
479          choice = ""
480          if choice == "local":
481              # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482              # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483              # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484              # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485              # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486              # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487              # below exactly as before (the weights-present check is the escape hatch).
488              if (

```

```

489         (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490         and _cloud_available(cfg)
491         and not _local_detector_weights_present(cfg, seg_name)
492     ):
493         return _failed(
494             f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495             "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
496             False,
497         )
498     return None # explicit "run on my machine" ? in-process regardless of the
server's target
499     if choice == "cloud":
500         if not _cloud_available(cfg):
501             return _failed(
502                 "cloud-serving was requested but this server isn't configured for it "
503                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                 False,
505             )
506         compute = get_compute_backend(cfg, target_override="cloud_run")
507     else:
508         compute = get_compute_backend(
509             cfg
510         ) # server default (default-OFF unless cloud_run)
511     if compute is None:
512         return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514     storage_cfg = getattr(cfg, "storage", None)
515
516     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517     try:
518         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519     except ValueError as e:
520         return _failed(
521             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522         )
523     if input_ref.backend == "local":
524         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530         # default, while a bare/local-looking root can't stage no matter what
input_backend says
531         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532         # gives us nothing to build the staged URI under).
533         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534         try:
535             root_backend = (
536                 storage.parse_uri(input_root).backend if input_root else "local"
537             )
538         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539             root_backend = "local"

```



```

540         if root_backend == "local":
541             return _failed(
542                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                 "Run job.",
545                 False,
546             )
547         local_src = input_ref.key # resolved local path of the upload
548         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549         try:
550             notify("staging upload to bucket")
551             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
553             storage.put(
554                 local_src,
555                 staged_uri,
556                 config=(
557                     storage_cfg.model_dump() if storage_cfg is not None else None
558                 ),
559             )
560         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561             logger.error(
562                 "[API] cloud-serving: failed to stage %s -> %s: %s",
563                 local_src,
564                 staged_uri,
565                 e,
566                 exc_info=True,
567             )
568             return _failed(
569                 f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570                 False,
571             )
572         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575         req.video_path = staged_uri
576         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577         db.add_video(
578             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579         )
580         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581         if not out_root:
582             return _failed(
583                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584                 "? that's where the Cloud Run job writes the result.",
585                 False,
586             )
587
588         notify("dispatching (cloud_run)")
589         logger.info(
590             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591             req.video_path,
592             out_root,
593         )

```

```

594         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598         overrides = []
599         idx_cfg = getattr(cfg, "indexing", None)
600         sk = getattr(idx_cfg, "skip_ai_handoff", None)
601         if sk is not None:
602             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
608         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609         wasb_cfg = getattr(idx_cfg, "wasb", None)
610         if wasb_cfg is not None:
611             decode_backend = getattr(wasb_cfg, "decode_backend", None)
612             if decode_backend:
613                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614             batch_size = getattr(wasb_cfg, "batch_size", None)
615             if batch_size:
616                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
617             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
618             if prefetch:
619                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
620         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
621         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
622         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
623         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
624         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
625         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
626         spec = ComputeJobSpec(
627             video_uri=req.video_path,
628             out_uri=out_root,
629             segmenter=seg_name,
630             config_overrides=tuple(overrides),
631             skip_validation=True,
632         )
633         try:
634             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
635             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
636             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
637             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
638             result = compute.run_infer(
639                 spec,
640                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
641             )

```

```

642         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
643             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
644             return _failed(f"cloud dispatch error: {e}", True)
645         if result.returncode != 0:
646             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
647         if result.video_id and result.video_id != video_id:
648             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
649             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
650             logger.warning(
651                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
652                 "and the worker hash; ingesting under the local id.",
653                 result.video_id,
654                 video_id,
655             )
656
657         notify("ingesting (cloud_run)")
658         storage_dict = (
659             storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
660             if hasattr(storage_cfg, "model_dump")
661             else {}
662         )
663         try:
664             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
665         except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
666             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
667             return _failed(
668                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
669                 True,
670             )
671
672         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
673         segments = [
674             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
675         ]
676         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
677         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
678         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
679         provenance = {
680             "path": "cloud_run",
681             "requested": req.compute,
682             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
683             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
684             "execution": result.execution,
685             "outputs_uri": result.outputs_uri,
686         }
687         logger.info(
688             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
689             result.execution,
690             provenance["job"],
691             provenance["region"],
692             result.outputs_uri,
693             video_id,
694         )
695         return (
696             {

```

```

697         "video_id": video_id,
698         "status": "success",
699         "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{{seg_name}}').",
700         "results": [r.model_dump() for r in val_results],
701         "play_segments": segments,
702         "compute": provenance,
703     },
704     True,
705 )
706
707
708 def _execute_processing(
709     config=None, db=None, req=None, progress=None
710 ) -> Dict[str, Any]:
711     """The full hash ? validate ? segment ? report pipeline for one video.
712
713     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714     module-level ``global_config``/``db_instance`` (backward-compatible with existing
715     tests that pass only ``req``). Route handlers always pass explicit params; older
716     test code that calls ``_execute_processing(req)`` still works.
717
718     ``progress`` is an optional callable(stage: str) used to surface coarse job
719     progress.
720
721     Raises on unexpected internal errors ? the caller records those as a job failure
722     """
723     import backend.main as _main # call-time seam: main's globals/model stay patchable
724
725     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
726     # shift positional args ? req arrives in position 0 (config), progress in position 1
727     (db).
728     if config is not None and not isinstance(config, AppConfig):
729         # Old signature: _execute_processing(req, progress=None)
730         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
731             req = db
732             db = None

```

12. user (2026-07-08T05:45:38.211Z)

```

730     elif req is None and isinstance(config, _main.ProcessRequest):
731         req = config
732         config = None
733     # Resolve config/db from module globals when not passed explicitly
734     if config is None:
735         config = _main.global_config
736     if db is None:
737         db = _main.db_instance
738
739     notify = progress or (lambda stage: None)
740
741     notify("hashing")
742     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
743     (identity ?
744     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
745     original
746     # req.video_path (the source URI) is kept for the DB record + report; only the file-
747     reading
748     # consumers (hash / validators / segmenter) use the resolved local path.
749     local_video = ""
750     try:
751         local_video = storage.acquire_local_input(
752             req.video_path, getattr(config, "storage", None)
753         )
754         video_id = _main.compute_video_id(local_video)
755     except Exception:

```

```

753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     try:
758         existing_video = db.get_video(video_id)
759     except Exception:
760         storage.cleanup_acquired_local_input(
761             req.video_path, local_video, getattr(config, "storage", None)
762         )
763         raise
764     was_reingest = existing_video is not None
765
766     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767     default_seg = getattr(
768         getattr(config, "indexing", None), "default_segmenter", "motion"
769     )
770     seg_name = req.segmenter or default_seg
771     segmenter_actual = None
772     segmentation_ran = False
773     val_results: List[Any] = []
774     val_context: Dict[str, Any] = {}
775     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
776     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
777     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
778     compute_actual: Optional[str] = None
779     response: Optional[Dict[str, Any]] = None
780     try:
781         if (
782             existing_video
783             and existing_video.get("status") == "processed"
784             and not req.force
785         ):
786             cached_segments = db.query_play_segments(video_id, {})
787             if cached_segments:
788                 logger.info(
789                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                     video_id,
791                 )
792             response = _cached_process_payload(
793                 video_id,
794                 existing_video.get("validation_results", []),
795                 cached_segments,
796                 getattr(req, "compute", None),
797             )
798             return response
799
800     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801     #     UNLESS this exact content already has a cached verdict under the current
validator config.
802     notify("validating")
803     # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804     # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805     # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806     # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807     # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation

```

```

passed but
808         # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809         # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810         # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811         # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812         # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813         # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814         val_fingerprint = registry.config_fingerprint(config)
815         # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
816         # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
817         # failing the job. The cache is a pure speedup ? never a new failure mode.
818         reused_validation = False
819         if not req.force:
820             try:
821                 cached = db.get_validation_cache(video_id)
822                 if cached is not None and cached.get("fingerprint") == val_fingerprint:
823                     # Replay the stored ValidationResult list; val_context stays {} (no
fresh
824                     # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
825                     # deliberate cost of skipping the decode). Segmentation reads
neither of these.
826                     val_results = [
827                         ValidationResult(**r) for r in cached.get("results", [])
828                     ]
829                     reused_validation = True
830                     logger.info(
831                         "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
832                         "validator config unchanged ? skipping the validator suite. "
833                         "force=true re-validates.",
834                         cached.get("passed"),
835                         video_id,
836                     )
837             except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
838                 reused_validation = False
839                 logger.warning(
840                     "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
841                     video_id,
842                     exc_info=True,
843                 )
844             if not reused_validation:
845                 logger.info(f"Running validations for {req.video_path}...")
846                 val_results, val_context = registry.run_all_with_context(local_video,
config)
847                 failures = [r for r in val_results if not r.passed]
848                 if not reused_validation:
849                     # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
850                     # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
851                 db.set_validation_cache(
852                     video_id,
853                     val_fingerprint,
854                     not failures,

```

```

855         [r.model_dump() for r in val_results],
856     )
857
858 if failures:
859     # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
860     # status; it no longer destroys the video's prior good segments.
861     db.add_video(
862         video_id,
863         req.video_path,
864         "failed",
865         [r.model_dump() for r in val_results],
866     )
867     response = {
868         "video_id": video_id,
869         "status": "failed",
870         "message": _validation_failure_message(failures),
871         "results": [r.model_dump() for r in val_results],
872     }
873     return response
874
875 # 2. Run segmenter & indexer
876 logger.info("[API] Video passed checks. Segmenting and indexing...")
877 db.add_video(
878     video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
879 )
880
881 segmenter_cls = segmenter_registry.get(seg_name)
882 if not segmenter_cls:
883     # Submit-time validation already 400s unknown names; this is the defensive
884     # in-worker path (e.g. config default pointing at an unregistered
segmenter).
885     available = list(segmenter_registry.list_available().keys())
886     response = {
887         "video_id": video_id,
888         "status": "failed",
889         "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
890         "results": [r.model_dump() for r in val_results],
891         "play_segments": [],
892     }
893     return response
894
895 logger.info(f"[API] Using segmenter: {seg_name}")
896 notify(f"segmenting ({seg_name})")
897 segmenter_actual = seg_name
898 # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
899 # if the run yields segments; otherwise drop the new partial rows and keep the
old.
900 play_wm, cand_wm = db.segment_watermarks(video_id)
901 # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
902 # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
903 # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
904 # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
905 # they're dropped on the cloud path in this slice ? a documented follow-up.)
906 remote = _maybe_dispatch_remote(
907     config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
908 )
909 if remote is not None:
910     response, segmentation_ran = (
911         remote # `ran` = did the cloud job actually execute (report)

```



```

966         "requested": getattr(req, "compute", None),
967     },
968 )
969 prov = response.get("compute", {}) if isinstance(response, dict) else {}
970 logger.info(
971     "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
972     video_id,
973     getattr(req, "compute", None),
974     prov.get("path"),
975     prov.get("dispatched"),
976     (response or {}).get("status") if isinstance(response, dict) else None,
977 )
978 # Always emit the per-video observability report (next to the video, or to
979 # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
980     # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
981     # doesn't surface arbitrary run signals.)
982     paths = _main.emit_indexer_report(
983         db=db,
984         video_id=video_id,
985         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
986         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
987         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
988         # still preserved in DB provenance via db.add_video.
989         video_path=local_video,

```

13. user (2026-07-08T05:45:38.691Z)

```

440     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
441     workspace_root: str = ""
442     # Cloud namespace prefix under the root so per-video folders sit tidily beside
443     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
444     workspace_prefix: str = "workspaces"
445     local: dict[str, Any] = Field(default_factory=dict)
446     s3: dict[str, Any] = Field(default_factory=dict)
447     gcs: dict[str, Any] = Field(default_factory=dict)
448     gdrive: dict[str, Any] = Field(default_factory=dict)
449     # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
450     # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451     gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
460         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
461         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
462         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.

```

```

464
465     Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472     # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473     # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474     # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475     # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476     # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477     max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478     # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
479     # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480     # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486     class BillingConfig(BaseModel):
487         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
493         inserts a real, calibrated version and points ``pricebook_version`` at it.
494
495         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
496         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
497         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
498
499         enabled: bool = False
500         pricebook_version: str = (
501             "placeholder-v0" # the seeded $0 version; owner flips to a real one
502         )
503         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
504         margin: float = 0.0 # resale markup (?0); 0 = bill at cost
505
506
507     class FlywheelConfig(BaseModel):
508         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
509         loop: on a CONSENTED adult job, durably **capture** the source + rally output into

```

```

the per-video
510         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
511         the owned model climbs toward the D11 ship floor.
512
513         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
514         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
515         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
516         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
517         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
518         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
519         owner/counsel/GPU scope. Pairs with M9-consent.""

```

14. user (2026-07-08T05:45:51.819Z)

```

140             continue # reject inf/nan times rather than persist a junk rally
141             meta: Dict[str, Any] = {
142                 "source": "cloud_run",
143                 "ending_reason": (row.get("ending_reason") or "").strip(),
144             }
145             sc = (row.get("shot_count") or "").strip()
146             if sc:
147                 try:
148                     meta["shot_count"] = int(
149                         float(sc)
150                     ) # OverflowError on 'inf'/'1e400'
151                 except (ValueError, OverflowError):
152                     meta["shot_count"] = sc
153             db.add_play_segment(video_id, start, end, metadata=meta)
154             n += 1
155         return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:

```

```

174         return True
175     idx = getattr(cfg, "indexing", None)
176     substrate = (
177         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178     ).strip().lower()
179     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181     else: # wasb_hybrid, or fusion on the (default) WASB substrate
182         path = (
183             os.environ.get("WASB_WEIGHTS")
184             or getattr(getattr(idx, "wasb", None), "weights_path", "")
185             or ""
186         )
187     return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190 def _cloud_available(cfg) -> bool:
191     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
192
193     True when the control plane is wired for Cloud Run: either the configured default
target is
194     already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195     (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196     are present so a forced per-request override can actually dispatch + collect a
result.
197
198     Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
199     deployment = getattr(cfg, "deployment", None)
200     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201         return True
202     has_job = bool(
203         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204     )
205     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206     return bool(has_job and out_root)
207
208
209 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
211     with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
212     (queue/cold-start), so the very first tick already shows movement."""
213     minutes = int(elapsed_sec) // 60
214     if minutes < 1:
215         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
216     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
217
218
219
220 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222     object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223     deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224     ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225     if getattr(input_ref, "backend", "") != "gcs":
226         return None

```

```

227     sc = _storage_settings(cfg)
228     if not sc:
229         return None
230     try:
231         st = storage.get_storage("gcs", config=sc).stat(input_ref)
232         return getattr(st, "md5", None) or None
233     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234         logger.info(
235             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236             getattr(input_ref, "uri", input_ref),
237             exc_info=True,
238         )
239     return None
240
241
242 def _cached_process_payload(
243     video_id: str, validation_results, segments, requested_compute
244 ) -> Dict[str, Any]:
245     """The /api/process result for a video whose detection already exists ? ONE shape
246     for
247     both producers (the drain's DB-cache short-circuit and the submit-time cache,
248 #484)."""
249     return {
250         "video_id": video_id,
251         "status": "success",
252         "message": (
253             f"Reused {len(segments)} cached play segments. "
254             "Pass force=true to reprocess this video."
255         ),
256         "results": validation_results or [],
257         "play_segments": segments,
258         "cached": True,
259         "compute": {"path": "cached", "requested": requested_compute},
260     }
261
262 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
263     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
264     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
265     decide."""
266     ref = storage.parse_uri(uri)
267     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
268         local = storage.fetch(
269             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
270         )
271         with open(local, encoding="utf-8") as f:
272             loaded = json.load(f)
273         return loaded if isinstance(loaded, dict) else None
274
275 # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
276 hits
277 # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
278 # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
279 instance
280 # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
281 _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
282 _REHYDRATE_LOCKS_GUARD = threading.Lock()
283
284 def _rehydrate_lock(video_id: str) -> threading.Lock:
285     with _REHYDRATE_LOCKS_GUARD:
286         return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())

```



```

342         video_id,
343         existing.get("validation_results", []),
344         segs,
345         requested_compute,
346     )
347     sc = _storage_settings(cfg)
348     if not sc or (sc.get("backend") or "local") == "local":
349         return None
350     output_root = (sc.get("output_root") or "").rstrip("/")
351     if not output_root:
352         return None
353     outputs_uri = f"{output_root}/outputs/{video_id}"
354     report_uri = f"{outputs_uri}/report.json"
355     if not storage.exists(report_uri, config=sc):
356         return None
357     report = _read_bucket_json(report_uri, sc)
358     if not report or report.get("status") != "success":
359         return None
360     if not video_path:
361         # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
362         # carries the worker's --video-uri (P3); older reports fall back to the
363         # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
364         # recover, and the next same-md5 submit re-binds the truthful path via
365         # add_video's upsert.
366         video_path = str(report.get("video_uri") or "") or _first_object_under(
367             f"{output_root}/videos/{video_id}/", sc
368         )
369     play_wm, cand_wm = db.segment_watermarks(video_id)
370     db.add_video(video_id, video_path, "processed", [])
371     n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
372     if n <= 0:
373         return None
374     segments = [
375         s
376         for s in db.query_play_segments(video_id, {})
377         if int(s.get("id", 0)) > play_wm
378     ]
379     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

```

15. user (2026-07-08T05:45:52.297Z)

```

1     """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3     Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4     - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5     for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6     so it is abandoned, not joined ? it never blocks the process.
7     - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8     ``max_wait_sec`` deadline).
9
10    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11    waiting (a fake clock advances instantly).
12
13    **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14    transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15    ``execution.policy`` logger to watch the poll trail.
16    """

```

```

17
18     from __future__ import annotations
19
20     import logging
21     import threading
22     import time as _time
23     from dataclasses import asdict, dataclass
24     from enum import Enum
25     from typing import Callable, Optional, TypeVar
26
27     T = TypeVar("T")
28
29     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30     LOG = logging.getLogger("execution.policy")
31
32
33     class WaitMode(str, Enum):
34         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
36         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37         ABORT = "abort" # one attempt; on hang/fail, stop immediately
38         BLOCK = "block" # bounded in-process blocking wait (no reschedule)
39
40
41     class PolicyTimeout(TimeoutError):
42         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45     @dataclass(frozen=True)
46     class ExecutionPolicy:
47         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48
49         mode: WaitMode = WaitMode.WAIT_AND_RESUME
50         poll_every_n_sec: float = 10.0
51         op_timeout_sec: float = (
52             120.0 # HARD per-attempt deadline ? the hang-killer that was missing
53         )
54         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
55         max_retries: int = 5 # retries (=> max_retries+1 attempts) for failures/hangs
56         backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
57         on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by P2/P3)
58         resumable: bool = True
59
60         def to_dict(self) -> dict:
61             d = asdict(self)
62             d["mode"] = self.mode.value
63             return d
64
65         @classmethod
66         def from_dict(cls, d: dict) -> "ExecutionPolicy":
67             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
68             kw = {k: v for k, v in d.items() if k in known}
69             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70                 kw["mode"] = WaitMode(kw["mode"])
71             return cls(**kw)
72
73
74     #: The sane default (also what a job gets if it declares no policy).
75     DEFAULT_POLICY = ExecutionPolicy()
76

```



```

77
78 def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79     """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80     return policy.backoff_base_sec * (2**attempt)
81
82
83 def run_bounded(
84     fn: Callable[[], T],
85     policy: ExecutionPolicy = DEFAULT_POLICY,
86     *,
87     label: str = "op",
88     logger: Optional[logging.Logger] = None,
89     sleep: Callable[[float], None] = _time.sleep,
90 ) -> T:
91     """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
92
93     Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
94     last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
95     """
96     log = logger or LOG
97     attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
98     last_exc: Optional[BaseException] = None
99     for attempt in range(attempts):
100         box: dict = {}
101         done = threading.Event()
102
103         # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
104         # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
105         def _runner(_box: dict = box, _done: threading.Event = done) -> None:
106             try:
107                 _box["val"] = fn()
108             except (
109                 BaseException
110             ) as e: # relay ANY error (incl. timeouts) to the caller thread
111                 _box["exc"] = e
112             finally:
113                 _done.set()
114
115         threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
116         if done.wait(timeout=policy.op_timeout_sec):
117             if "exc" in box:
118                 last_exc = box["exc"]
119                 log.info(
120                     "%s: attempt %d/%d failed: %s",
121                     label,
122                     attempt + 1,
123                     attempts,
124                     last_exc,
125                 )
126             else:
127                 if attempt:
128                     log.info(
129                         "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
130                     )
131                 return box["val"] # type: ignore[return-value]
132         else:
133             last_exc = PolicyTimeout(
134                 f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
135             )
136             log.warning(

```

```

137         "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
138         label,
139         attempt + 1,
140         attempts,
141         policy.op_timeout_sec,
142     )
143     if attempt < attempts - 1:
144         wait = _backoff_sec(policy, attempt)
145         log.debug("%s: backing off %.1fs before retry", label, wait)
146         sleep(wait)
147     assert last_exc is not None
148     log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
149     raise last_exc
150
151
152     def poll_until(
153         check: Callable[[], Optional[T]],
154         policy: ExecutionPolicy = DEFAULT_POLICY,
155         *,
156         label: str = "op",
157         logger: Optional[logging.Logger] = None,
158         sleep: Callable[[float], None] = _time.sleep,
159         monotonic: Callable[[], float] = _time.monotonic,
160         on_tick: Optional[Callable[[float], None]] = None,
161     ) -> T:
162         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
163         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
164
165         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
166         ``on_tick(elapsed_sec)`` (optional) fires once per still-waiting iteration ? the
caller's live
167         progress heartbeat (#482). Best-effort by contract: a raising callback is swallowed
168         (DEBUG-logged), never allowed to kill the wait it reports on.
169         """
170         log = logger or LOG
171         start = monotonic()
172         polls = 0
173         log.info(
174             "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
175             label,
176             policy.poll_every_n_sec,
177             policy.max_wait_sec,
178         )
179         while True:
180             result = check()
181             if result is not None:
182                 log.info(
183                     "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
184                 )
185                 return result
186             elapsed = monotonic() - start
187             if elapsed >= policy.max_wait_sec:
188                 log.warning(
189                     "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
190                 )
191                 raise PolicyTimeout(
192                     f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
193                 )
194             polls += 1
195             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
196             if on_tick is not None:
197                 try:

```

```

198         on_tick(elapsed)
199     except Exception: # noqa: BLE001 ? progress reporting must never kill the
wait
200         log.debug("%s: on_tick callback failed", label, exc_info=True)
201         sleep(policy.poll_every_n_sec)
202

```

16. user (2026-07-08T05:46:02.063Z)

```

1     """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3     The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4     Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5     and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6     Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8     SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9     monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import ValidationResult, registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
43         from backend.main import ProcessRequest # noqa: F401
44

```

```

45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
48     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
61         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62         """
63         if segments:
64             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65         else:
66             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69     def _validation_failure_message(failures: List[Any]) -> str:
70         details = []
71         for failure in failures:
72             name = getattr(failure, "validator_name", None) or "validation"
73             message = (getattr(failure, "message", None) or "").strip()
74             details.append(f"{name}: {message}" if message else str(name))
75         if not details:
76             return "Video failed validation checks."
77         return "Video failed validation: " + "; ".join(details)
78
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
scratch."""
82         storage_config = (
83             config.get("storage")
84             if isinstance(config, dict)
85             else getattr(config, "storage", None)
86         )
87         try:
88             if storage.input_is_local(raw_input, storage_config):
89                 return config
90         except ValueError:
91             return config
92         try:
93             cfg = (
94                 config.model_dump(mode="json")
95                 if hasattr(config, "model_dump")
96                 else dict(config or {})
97             )
98         except Exception:
99             cfg = {}
100         report = dict(cfg.get("report") or {})
101         if report.get("output_dir"):

```

```

102         return config
103     report["output_dir"] = os.path.join(output_base(config), "reports")
104     cfg["report"] = report
105     return cfg
106
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
112         the bucket and
113         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
114         call ? so
115         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
116         were produced.
117         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
118         pattern (cloud
119         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
120         malformed row is
121         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
122         temp dir is
123         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
124         a job failure
125         and prunes any partial rows (destroy-AFTER-confirm)."""
126         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
127         ref = storage.parse_uri(uri)
128         n = 0
129         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
130             local = storage.fetch(
131                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
132             )
133             with open(local, newline="", encoding="utf-8") as f:
134                 for row in csv.DictReader(f):
135                     if n >= max_rows:
136                         logger.warning(
137                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
138                             rest.",
139                             max_rows,
140                             uri,
141                         )
142                     break
143             try:
144                 start, end = float(row["start"]), float(row["end"])
145             except (KeyError, TypeError, ValueError):
146                 continue
147             if not (math.isfinite(start) and math.isfinite(end)):
148                 continue # reject inf/nan times rather than persist a junk rally

```

17. user (2026-07-08T05:46:03.533Z)

```

deploy\gcp\teardown.sh:29:SA_EMAIL="rally-worker@${PROJECT}.iam.gserviceaccount.com"
deploy\gcp\provision.sh:18:SA_NAME="rally-worker"
deploy\gcp\provision.sh:20:SA_KEY="${RALLY_SA_KEY:-$HOME/.gcp/rally-worker-sa.json}"
deploy\gcp\create_gpu_vm.sh:4:# rally-worker service account (+ cloud-platform scope so it can
read/write the bucket + report
deploy\gcp\create_gpu_vm.sh:25:SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
deploy\gcp\README.md:35:| Service account | `rally-worker@gen-lang-
client-0306026826.iam.gserviceaccount.com` (`roles/storage.objectAdmin`) | $0 | `gcloud iam
service-accounts delete` |
deploy\gcp\README.md:36:| SA key | `~/.gcp/rally-worker-sa.json` (local only, **never**
committed) | $0 | delete the file + the key |
deploy\gcp\README.md:92:> L4 capacity (Mumbai ? Singapore), uses the driver-ready image,
attaches the `rally-worker` SA, and
deploy\gcp\README.md:113: --service-account=rally-worker@gen-lang-
client-0306026826.iam.gserviceaccount.com \

```

```

deploy\gcp\README.md:132:P=gen-lang-client-0306026826; SA=rally-
worker@${PROJECT}.iam.gserviceaccount.com
deploy\gcp\README.md:136:# 2. let rally-worker publish metrics + logs:
deploy\gcp\README.md:185:gcloud compute scp ~/.gcp/rally-worker-sa.json rally-
gpu:/root/.gcp/sa.json --zone=asia-south1-a
deploy\cloudrun\cloudbuild.yaml:13: _IMAGE: rally-worker
deploy\gcp\nightly_regression\launch.sh:24:SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
deploy\gcp\nightly_regression\register_scheduler.sh:26:SA="rally-
worker@${PROJECT}.iam.gserviceaccount.com"
deploy\gcp\nightly_regression\register_scheduler.sh:57: --image="$IMAGE" --service-
account="$SA" --max-retries=0 --task-timeout=900s \
deploy\gcp\nightly_regression\create_nightly_bucket.sh:23:SA="rally-
worker@${PROJECT}.iam.gserviceaccount.com"
deploy\cloudrun\gen_service_yaml.py:29: with it; the code default ("rally-worker") is already
correct.
deploy\cloudrun\gen_service_yaml.py:156: help="Container image; default = the project's
rally-worker:latest (post-#470 it "
deploy\cloudrun\gen_service_yaml.py:193: args.image =
f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-worker:latest"
deploy\cloudrun\Dockerfile:122:# infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:///
--segmenter wasb_hybrid --set ?
deploy\cloudrun\post_deploy_smoke.ps1:3: Post-deploy smoke for the rally-worker Cloud Run Job:
dispatch ONE clip, wait, and surface
deploy\cloudrun\post_deploy_smoke.ps1:24: [string]$Job = "rally-worker",
deploy\cloudrun\README.md:32:gcloud run jobs create rally-worker \
deploy\cloudrun\README.md:36: --max-retries 0 --task-timeout 3600 \
deploy\cloudrun\README.md:71:gcloud run jobs update rally-worker --region $REGION --update-env-
vars WASB_DECODE_BACKEND=nvdec_resident
deploy\cloudrun\README.md:72:gcloud run jobs update rally-worker --region $REGION --remove-env-
vars WASB_DECODE_BACKEND # ? forwarded/preset default
deploy\cloudrun\README.md:74:gcloud run jobs update rally-worker --region $REGION --image
$REGION-docker.pkg.dev/$PROJECT/$REPO/rally-worker@sha256:<prev>
deploy\cloudrun\README.md:104:export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
deploy\gcp\nightly_regression\setup_email.sh:20:SA="rally-
worker@${PROJECT}.iam.gserviceaccount.com"

```

18. assistant (2026-07-08T05:46:10.912Z)

Now let me look at the job worker (how failures surface to jobs) and the upload endpoint's streamed hash.

19. user (2026-07-08T05:46:12.349Z)

```

100         )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
106         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
107         NOT a failure. The worker persists `next_poll_at` + `progress` (via
108         `set_job_waiting`),
109         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
110         date
111         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
112         contract.
113
114         The wiring is additive: the production segmenter path never raises this today (zero
115         behaviour change), so a job runs exactly as before. The hook is what a later slice
116         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
117         """
118
119     def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
120         super().__init__(f"job parked for {delay_sec:.0f}s")

```

```

118         self.delay_sec = max(0.0, float(delay_sec))
119         self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
138             # maybe_dispatch_remote, which
139             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
140             # only read off
141             # the job row for analytics). NULL ? server default = pre-picker behaviour.
142             compute=job.get("compute"),
143             force=bool(params.get("force")),
144         )
145
146     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
147         """Run ONE job already claimed (`status='running`) by `claim_due_job`, recording
148         its
149         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
150
151         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
152         means
153         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
154         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
155         the
156         job self-scheduled and will be re-claimed when `next_poll_at` is due.
157         """
158         import backend.main as _main
159
160         job_id = job["id"]
161         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
162         (NULL/default ? the
163         # original /api/process pipeline). Both record their terminal state the same way;
164         only process
165         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
166         usage/segments).
167         is_compile = job.get("kind") == "compile"
168         try:
169             if is_compile:
170                 result = _main._execute_compile(
171                     config,
172                     db,
173                     job,
174                     progress=lambda stage: db.set_job_progress(job_id, stage),
175                 )
176             else:
177                 req = _main._job_to_request(job)
178                 result = _main._execute_processing(
179                     config,
180                     db,
181                     req,
182                 )

```

```

175         progress=lambda stage: db.set_job_progress(job_id, stage),
176     )
177     if result.get("video_id"):
178         db.set_job_video_id(job_id, result["video_id"])
179     db.mark_job_done(job_id, result)
180     if not is_compile:
181         _maybe_record_job_cost(
182             job, result, config, db
183         ) # M8 cost ledger (default-OFF; process jobs only)
184         _maybe_run_flywheel(
185             job, result, config
186         ) # M11 data-flywheel (process jobs only)
187 except _main.JobWaiting as w:
188     # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189     logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190     db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191 except Exception as e:
192     logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193     db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196 def _maybe_record_job_cost(
197     job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198 ) -> None:
199     """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200     ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201     (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
202     pricebook, and
203     persist it.
204
205     ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
206     so even with
207     ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
208     **0** until a
209     FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
210     ledger + pricebook
211     + the recording seam; the usage-emission step is separate (the metering hooks on the
212     real run).
213     With the placeholder pricebook the cost is 0 regardless.
214
215     Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
216     is the
217     claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
218
219     billing_cfg = getattr(config, "billing", None)
220     if not billing_cfg or not getattr(billing_cfg, "enabled", False):
221         return
222     try:
223         usage = (result or {}).get("usage") or {}
224         db.record_job_cost(
225             job["id"],
226             usage=usage,
227             pricebook_version=billing_cfg.pricebook_version,
228             gpu_type=usage.get("gpu_type"),
229             compute_backend=(result or {}).get("compute_backend"),
230             owner_id=job.get("owner_id"),
231             margin=billing_cfg.margin,
232         )
233         logger.info(
234             "Recorded cost for job %s (pricebook=%s).",
235             job["id"],
236             billing_cfg.pricebook_version,
237         )
238     except Exception as e: # never wedge a completed job on bookkeeping
239         logger.warning(

```



```

234         "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
235     )
236
237
238     def _maybe_run_flywheel(
239         job: Dict[str, Any], result: Dict[str, Any], config: Any
240     ) -> None:
241         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
242         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
243         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
244         land), so this captures **nothing** in production. When activated it captures the
consented job's
245         artifacts into the per-video workspace (? later labeling ? the golden training set).
246
247         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
248         row (carries ``owner_id``); ``result`` is the pipeline output."""
249
250         flywheel_cfg = getattr(config, "flywheel", None)
251         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
252             return
253         try:
254             from backend import flywheel
255
256             flywheel.run_for_job(job, result, config)
257         except Exception as e: # never wedge a completed job on the flywheel
258             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
259
260
261     def _drain_due_jobs(config: Any, db: Any) -> int:
262         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
263         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
264
265         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
266         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267         a single-worker box keeps making progress with no central timer (the DB is the
clock).
268         Returns the number of jobs that reached a terminal/parked state this tick.
269         """
270         import backend.main as _main
271
272         ran = 0
273         while True:
274             job = db.claim_due_job()
275             if job is None:
276                 break
277             ran += 1
278             _main._run_claimed_job(job, config, db)
279         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new

```

20. user (2026-07-08T05:46:13.744Z)

```

backend\api\uploads.py:15: ``input_root``) plus the streamed md5 as ``video_id``.
backend\api\uploads.py:43:from backend.utils.hashing import compute_video_id # canonical file-
identity hashing
backend\api\uploads.py:120:         # A streaming writer has no cancel ? close() finalizes
whatever was staged, then
backend\api\uploads.py:171:         # Remote staging: stream parts straight into the bucket under

```

```

the input root ?
backend\api\uploads.py:179:         staging = None # backend can't stream (e.g. mounted-
Drive) ? stage locally
backend\api\uploads.py:185:         "hasher": hashlib.md5(),
backend\api\uploads.py:235:         # oversize stream), then hand it to the blocking bucket
writer in one threadpool
backend\api\uploads.py:238:         async for chunk in request.stream():
backend\api\uploads.py:257:         async for chunk in request.stream():
backend\api\uploads.py:288:         """Finalize: verify part count, move into place, return {path,
video_id} for /api/process."""
backend\api\uploads.py:300:         # Remote staging: close() finalizes the streamed object; the
md5 accumulated per
backend\api\uploads.py:301:         # part IS compute_video_id's digest (same bytes, same
algorithm), so /api/process
backend\api\uploads.py:316:         "video_id": st["hasher"].hexdigest(),
backend\api\uploads.py:329:         video_id = compute_video_id(final_path)
backend\api\uploads.py:332:         "video_id": video_id,

```

21. user (2026-07-08T05:46:23.488Z)

```

160         status_code=400,
161         detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
162     )
163     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
164         raise HTTPException(
165             status_code=413,
166             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
167         )
168     upload_id = uuid.uuid4().hex
169     staging = _staging_target()
170     if staging is not None:
171         # Remote staging: stream parts straight into the bucket under the input root ?
172         # never assembled on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2). The
173         # upload_id in the key keeps concurrent same-name uploads distinct, and the
free-
174         # space guard is skipped: no local bytes are consumed.
175         stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"
176         try:
177             writer = storage.open_writer(stage_uri, config=staging["config"])
178         except NotImplementedError:
179             staging = None # backend can't stream (e.g. mounted-Drive) ? stage locally
180         else:
181             with _uploads_lock:
182                 _uploads[upload_id] = {
183                     "filename": name,
184                     "writer": writer,
185                     "hasher": hashlib.md5(),
186                     "stage_uri": stage_uri,
187                     "config": staging["config"],
188                     "parts": 0,
189                     "bytes": 0,
190                 }
191             return {
192                 "upload_id": upload_id,
193                 "max_part_mb": part_bytes // (1024 * 1024),
194                 "next_part": f"/api/upload/{upload_id}/part/0",
195             }
196     os.makedirs(upload_dir, exist_ok=True)
197     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
effort ? an
198     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
199     # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
200     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")

```

```

201     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
202     open(tmp_path, "wb").close()
203     with _uploads_lock:
204         _uploads[upload_id] = {
205             "filename": name,
206             "tmp_path": tmp_path,
207             "parts": 0,
208             "bytes": 0,
209         }
210     return {
211         "upload_id": upload_id,
212         "max_part_mb": part_bytes // (1024 * 1024),
213         "next_part": f"/api/upload/{upload_id}/part/0",
214     }
215
216
217 @router.put("/api/upload/{upload_id}/part/{index}")
218 async def upload_part(upload_id: str, index: int, request: Request):
219     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
220     _, max_bytes, part_bytes, _ = _upload_cfg()
221     with _uploads_lock:
222         st = _uploads.get(upload_id)
223         if st is None:
224             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
225         if index != st["parts"]:
226             raise HTTPException(
227                 status_code=409,
228                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
229             )
230
231         received = 0
232         overflow = None
233         if st.get("writer") is not None:
234             # Remote staging: buffer THIS part (? max_part_mb ? the caps above still abort
an
235             # oversize stream), then hand it to the blocking bucket writer in one threadpool
236             # hop so a network flush never stalls the event loop mid-upload.
237             buf = bytearray()
238             async for chunk in request.stream():
239                 received += len(chunk)
240                 if received > part_bytes:
241                     overflow = (
242                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
243                     )
244                     break
245                 if st["bytes"] + received > max_bytes:
246                     overflow = (
247                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
248                     )
249                     break
250                 buf += chunk
251             if not overflow and buf:
252                 data = bytes(buf)
253                 await run_in_threadpool(st["writer"].write, data)
254                 st["hasher"].update(data)
255         else:
256             with open(st["tmp_path"], "ab") as fh:
257                 async for chunk in request.stream():
258                     received += len(chunk)
259                     if received > part_bytes:
260                         overflow = (
261                             f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
262                         )

```

```

263             break
264         if st["bytes"] + received > max_bytes:
265             overflow = (
266                 f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
267             )
268             break
269             fh.write(chunk)
270     if overflow:
271         _abort_upload(upload_id)
272         raise HTTPException(
273             status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
274         )
275
276     with _uploads_lock:
277         st["parts"] += 1
278         st["bytes"] += received
279     return {
280         "upload_id": upload_id,
281         "received_parts": st["parts"],
282         "received_bytes": st["bytes"],
283     }
284
285
286     @router.post("/api/upload/{upload_id}/complete")
287     def upload_complete(upload_id: str, req: UploadCompleteRequest):
288         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
289         upload_dir, _, _, _ = _upload_cfg()
290         with _uploads_lock:
291             st = _uploads.get(upload_id)
292         if st is None:
293             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
294         if st["parts"] == 0 or st["parts"] != req.total_parts:
295             raise HTTPException(
296                 status_code=409,
297                 detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
298             )
299         if st.get("writer") is not None:
300             # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301             # part IS compute_video_id's digest (same bytes, same algorithm), so
/api/process
302             # gets the identity without anyone re-reading the video. (Sync endpoint ?
FastAPI
303             # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
304             try:
305                 st["writer"].close()
306             except Exception as e: # noqa: BLE001 ? surface a clean verdict, then clean up
307                 _abort_upload(upload_id)
308                 raise HTTPException(
309                     status_code=502,
310                     detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",
311                 ) from e
312         with _uploads_lock:
313             _uploads.pop(upload_id, None)
314         return {
315             "path": st["stage_uri"],
316             "video_id": st["hasher"].hexdigest(),
317             "size_bytes": st["bytes"],
318             "next": "POST /api/process with this path",
319         }
320     final_path = os.path.join(upload_dir, st["filename"])
321     if os.path.exists(
322         final_path
323     ): # never overwrite an earlier upload with the same name

```

```

324         stem, ext = os.path.splitext(st["filename"])
325         final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
326         os.replace(st["tmp_path"], final_path)
327         with _uploads_lock:
328             _uploads.pop(upload_id, None)
329         video_id = compute_video_id(final_path)
330         return {
331             "path": os.path.abspath(final_path),
332             "video_id": video_id,
333             "size_bytes": os.path.getsize(final_path),
334             "next": "POST /api/process with this path",
335         }
336
337
338     @router.delete("/api/upload/{upload_id}")
339     def upload_abort(upload_id: str):

```

22. user (2026-07-08T05:46:24.151Z)

```

100     # edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-
plane
101     # deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT
overlay).
102     # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
103     # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
104     # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
105     # single-sourced in requirements-trackers.txt.
106     RUN python3 -m pip install --upgrade pip \
107         && python3 -m pip install -r requirements-trackers.txt \
108         && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
109
110     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
111     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
112     # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
113     # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
114     # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
115     ENV PYTHONUNBUFFERED=1 \
116         WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
117         WASB_REPO_DIR=/opt/models/WASB-SBDT \
118         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
119         RALLY_DETECTOR_IMPL=native
120
121     # The dispatcher (CloudRunCompute) supplies the per-execution args:
122     # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
123     ENTRYPOINT ["python3", "-m", "backend.worker"]
124

```

23. assistant (2026-07-08T05:46:47.867Z)

A few final details ? `acquire\_local\_input`, and how the submit path uses
`submit\_time\_video\_id`.

24. user (2026-07-08T05:46:48.529Z)

backend\storage__init__.py:53: def resolve_input_ref(raw: str, storage_config: Any = None) ->
StorageRef:

```

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-54-    """Resolve a user-supplied input (a path or storage
URI) into a ``StorageRef`` under the input
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-55-    config (``input_backend``/``input_root``). Raises
``ValueError`` for an unsupported URI scheme ?
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-56-    callers at the API/CLI boundary catch that and return
a clean client error (never a 500).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-57-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-58-    Use ``ref.backend`` to branch local-vs-remote and
``ref.key`` for the resolved LOCAL path (e.g.
backend\storage\__init__.py-59-    ``local://?`` is stripped to the bare path), rather than
stat-ing the raw URI string."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-60-    sc = _storage_dict(storage_config)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-61-    uri = resolve_input_uri(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-62-        raw,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-63-        input_backend=sc.get("input_backend", "local"),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-64-        input_root=sc.get("input_root", ""),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-65-    )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-66-    return parse_uri(uri)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-67-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-68-
backend\storage\__init__.py:69:def input_is_local(raw: str, storage_config: Any = None) -> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-70-    """True if ``raw`` resolves (under the input config)
to a LOCAL filesystem path ? i.e. a
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-71-    plain on-disk file the caller can stat/validate
directly. False for a bucket / Drive URI
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-72-    that must be fetched first. Raises ``ValueError`` for
an unsupported scheme (same as
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-73-    ``resolve_input_ref``)."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-74-    return resolve_input_ref(raw, storage_config).backend
=="local"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-75-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-76-
backend\storage\__init__.py:77:def acquire_local_input(
backend\storage\__init__.py-78-    raw: str, storage_config: Any = None, *, scratch_parent: "str
| None" = None
backend\storage\__init__.py-79-) -> str:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-80-    """Resolve ``raw`` (a path or storage URI) to a LOCAL
file path ready for processing.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-81-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-82-    Local / bare paths are returned UNCHANGED (identity
passthrough, no copy ? today's behaviour

```

```

backend\storage\__init__.py-83-    byte-for-byte). A bucket \ Drive source
(``gs://``s3://``gdrive://``, or a bare name under a
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-84-    non-local ``input_root``/``input_backend``) is
downloaded into a fresh scratch dir and the local
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-85-    path returned. The caller owns successful remote
scratch copies; use
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-86-    ``cleanup_acquired_local_input`` when the consumer
has finished reading the returned path."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-87-    sc = _storage_dict(storage_config)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-88-    ref = resolve_input_ref(raw, sc)
backend\storage\__init__.py-89-    if ref.backend == "local":
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-90-        return (
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-91-            ref.key
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-92-        ) # identity ? no temp, no copy, byte-identical
to passing the path directly
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-93-    scratch = tempfile.mkdtemp(prefix="rally-input-",
dir=scratch_parent)
backend\storage\__init__.py-94-    try:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-95-        return fetch(ref.uri, os.path.join(scratch,
ref.name or "in.mp4"), config=sc)
backend\storage\__init__.py-96-    except Exception:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-97-        shutil.rmtree(scratch, ignore_errors=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-98-        raise
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-99-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-100-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-101-def cleanup_acquired_local_input(
backend\storage\__init__.py-102-    raw: str, local_path: str, storage_config: Any = None

```

25. user (2026-07-08T05:46:49.116Z)

```

236     _CompileError,
237     _execute_compile,
238     _execute_processing,
239     _finalize_reingest,
240     _ingest_remote_segments,
241     _maybe_dispatch_remote,
242     _remote_input_size_bytes,
243     _report_config_for_input,
244     _resolve_compile,
245     read_path_rehydrate,
246     signed_reel_url,
247 )
248
249
250 # Legacy thin wrapper for any remaining direct calls during transition.
251 # New code should import load_app_config / AppConfig from backend.config directly.
252 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
253     cfg = load_app_config(config_path)
254     return cfg.model_dump(mode="json")
255

```

```

256
257 # --- API Models ---
258 class ProcessRequest(BaseModel):
259     video_path: str
260     player_pool: Optional[List[str]] = None
261     max_frames: Optional[int] = None
262     segmenter: Optional[str] = None
263     # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
264     # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265     locale: Optional[str] = None
266     # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267     # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268     # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269     compute: Optional[str] = None
270     # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271     # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
272     force: bool = False
273
274
275 class AnnotationsRequest(BaseModel):
276     # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
277     # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
278     video_id: str
279     csv: str
280
281
282 class AssetRequest(BaseModel):
283     # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
284     #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
285     #   local working copy for the run), OR
286     #   source_uri ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
287     #   registered as-is, no copy (D14b).
288     # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
289     # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
290     uploaded_path: Optional[str] = None
291     source_uri: Optional[str] = None
292     medium_uri: str
293     sport: Optional[str] = None
294
295
296 class QueryRequest(BaseModel):
297     video_id: str
298     server: Optional[str] = None
299     receiver: Optional[str] = None
300     duration_min: Optional[float] = None
301     event_type: Optional[str] = None
302
303
304 class CompileRequest(BaseModel):
305     video_id: str
306     segment_ids: List[int]

```



```

307         output_filename: str
308         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
309         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
310         locale: Optional[str] = None
311
312
313     # --- API Endpoints ---
314     @app.get("/api/config")
315     def get_config(request: Request):
316         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
317         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
318         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
319         # by tests/test_redact.py + the /api/config redaction test).
320         cfg = request.app.state.config
321         if cfg is None:
322             return {}
323         return redact_secrets(cfg.model_dump(mode="json"))
324
325
326     @app.get("/api/health")
327     def health(request: Request):
328         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
329         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
330         confirm the engine is reachable and which detector is wired. Never raises."""
331         cfg = request.app.state.config
332         sport = getattr(cfg, "sport", None)
333         indexing = getattr(cfg, "indexing", None)
334         return {
335             "status": "ok",
336             "sport": sport,
337             "default_segmenter": getattr(indexing, "default_segmenter", None),
338         }
339
340
341     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
342     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
343     API_VERSION = 1
344
345

```

26. assistant (2026-07-08T05:46:56.110Z)

One last check ? where `submit\_time\_video\_id` is consumed on the submit path.

27. user (2026-07-08T05:46:57.962Z)

```

backend\api\jobs.py:76:     need = orchestration._remote_input_size_bytes(input_ref, getattr(cfg,
"storage", None))
backend\api\jobs.py:98:     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
backend\main.py:242:     _remote_input_size_bytes,
backend\main.py:574:     need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage",
None))
backend\main.py:803:     need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage",
None))
backend\orchestration.py:220: def submit_time_video_id(input_ref, cfg) -> Optional[str]:
backend\orchestration.py:1005: def _remote_input_size_bytes(input_ref, storage_config) ->
Optional[int]:

```

28. user (2026-07-08T05:47:04.588Z)

```
55     if claimed["created"]:
56         return (True, None)
57     return (
58         False,
59         JSONResponse(
60             status_code=202,
61             content={
62                 "job_id": claimed["job_id"],
63                 "status": claimed["status"],
64                 "poll": f"/api/jobs/{claimed['job_id']}",
65                 "deduplicated": True,
66             },
67         ),
68     )
69
70
71 def _fetch_scratch_guard(cfg: Any, input_ref: Any) -> None:
72     """P2/D11 507 fast-fail for a remote input the DRAIN will fetch to local scratch
73     (hash + validate). Raises ``HTTPException(507)`` when the box can't hold the
74     copy."""
75     from backend import orchestration # call-time seam (mirrors submit_process_job)
76     need = orchestration._remote_input_size_bytes(input_ref, getattr(cfg, "storage",
77     None))
78     require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
79
80 def submit_process_job(
81     db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
82 ) -> Optional[Tuple[bool, JSONResponse]]:
83     """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in
84     one hook.
85
86     Order matters (#492 review):
87     1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side
88     effects.
89     2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
90     fetches to local scratch, so low temp space must not reject it (review blocker
91     2).
92     3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
93     re-hydrate side effect, so the first request owns the cache completion (blocker
94     1).
95     4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself
96     serialized
97     per video_id) and completes the job instantly: no drain, no GPU dispatch.
98
99     Returns ``(queued, response)``: ``queued`` True ? the caller kicks the drain;
100     ``None`` ?
101
102     DB failure (the caller 500s)."""
103     from backend import orchestration # call-time seam: tests patch orchestration attrs
104
105     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
106     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
107         db, cfg, video_id
108     )
109     if input_ref.backend != "local" and not probed:
110         _fetch_scratch_guard(cfg, input_ref) # raises 507 pre-claim, exactly as before
111     outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
112     if outcome is None:
113         return None
114     created, dedup_response = outcome
115     if not created:
116         assert dedup_response is not None # claim contract: not-created ? dedup
```

```

response
110         return (False, dedup_response)
111     if probed:
112         cached = orchestration.cached_process_result(
113             db, cfg, video_id, req.video_path, requested_compute=req.compute
114         )
115         if cached is not None:
116             db.mark_job_done(job_id, cached)
117             return (
118                 False,
119                 JSONResponse(
120                     status_code=202,
121                     content={
122                         "job_id": job_id,
123                         "status": "done",
124                         "poll": f"/api/jobs/{job_id}",
125                         "cached": True,
126                     },
127                 ),
128             )
129     # The probe said hit but realizing it failed (report vanished / corrupt csv):
130     fall
131     # back to the normal queued dispatch ? but the scratch guard was skipped above,
132     so it
133     # must run NOW. If it fails, the already-claimed row is marked failed BEFORE
134     raising
135     # (a bare raise would leave a phantom 'queued' job for the drain to grab).
136     if input_ref.backend != "local":
137         try:
138             _fetch_scratch_guard(cfg, input_ref)
139         except HTTPException:
140             db.mark_job_failed(
141                 job_id,
142                 "cache re-hydrate failed and local scratch cannot hold a fetch of "
143                 "the source; free disk space or retry later",
144             )
145         raise
146     return (
147         True,

```

29. assistant (2026-07-08T05:48:20.659Z)

I have the full picture now. Let me compile the structured map.

30. user (2026-07-08T05:50:09.586Z)

Structured output provided successfully